MiniShop S302 - préparation LineServ, recette pré-démo et retour à la démo

> **État au moment de la rédaction : plan documentaire, pas déploiement ou recette validé.**
> Aucun accès à LineServ, test de navigateur réel, écriture ou réinitialisation de la base de production n'a été effectué depuis ce dépôt. Les **164 scénarios et 36 couples de transition** (200 lignes de suivi) restent `NON JOUÉ`. La cible manuelle prévue est la base réellement utilisée par l'application de production LineServ ; après les tests, cette même base doit être restaurée à l'état de démonstration avant la démo. `minishop_test` reste réservée aux tests automatisés GitHub CI.

> Ce guide reprend les indications communiquées pour le cours : SFTP `linserv-info-01.campus.unice.fr`, fichiers dans `www/`, URL en `https://linserv-info-01.campus.unice.fr/~login/...`, phpMyAdmin à `https://linserv-info-01.campus.unice.fr/phpmyadmin`. La disponibilité, les droits et la configuration actuels n'ont **pas** été vérifiés. Suivre les consignes de l'IUT si elles diffèrent.

1. Décision indispensable avant la mise en ligne

L'exemple visé pour ce compte est `https://linserv-info-01.campus.unice.fr/~pj509414/s302/`. **Ne pas considérer cette URL comme fonctionnelle ou publiée.** La procédure simple d'un fichier `sqlconnect.php` de TD ne suffit pas pour cette application :

1. **Sous-répertoire : non pris en charge en l'état.** `public/.htaccess` contient `RewriteBase /`, le routeur traite le chemin depuis la racine et des liens, formulaires, scripts et redirections commencent par `/`. Depuis `/~pj509414/s302/`, ils peuvent sortir du dossier, casser les images et les routes ou rediriger ailleurs. Il faut adapter et tester le préfixe pour l'ensemble de ces usages ; ne pas simplement changer `RewriteBase`.
2. **Racine publique : seul `public/` doit être servi.** `public/index.php` charge `../app/bootstrap.php` et suppose que `app/` est à côté de `public/`. Copié seul dans `www/s302/`, il ne trouve donc pas l'application. Copier l'ensemble du projet dans `www/s302/` exposerait potentiellement `.env`, données, code et documents ; les protections Apache ne sont pas garanties. **Ne pas le faire.** Il faut une séparation validée : application et `.env` hors de `www/`, `public/` seul accessible, avec chemin de bootstrap et configuration Apache adaptés ; ou un hébergement dédié dont le DocumentRoot pointe vers `public/` et qui accepte les routes de l'application.
3. **Sécurité de la recette :** restreindre l'accès au site de test avant toute exposition, car les comptes du jeu SQL ont des mots de passe publics connus. L'accès SFTP ne protège pas les pages HTTP. Confirmer avec l'IUT si un contrôle d'accès est possible et comment il est appliqué. Ne pas publier un admin de démonstration accessible à tous.
4. **Pré-requis à confirmer :** PHP Web >= 8.2 avec `pdo_mysql`, `mbstring`, `json`, `intl` ; MariaDB >= 10.6 ou MySQL >= 8.0.16 ; `mod_rewrite` et `.htaccess` autorisés ; droits applicatifs minimaux sur la base de production attribuée et autorisation de sauvegarde/restauration ; système de fichiers et session PHP utilisables. Le PHP en ligne de commande et celui du serveur web peuvent différer.

Point d'arrêt : tant que les points 1 à 4 ne sont pas levés, préparer les fichiers localement

seulement ; ne pas lancer la campagne, ne pas créer de base manuelle distincte et ne pas présenter le site comme disponible. Aucune commande d'installation système, `sudo`, `npm install`, ouverture de port ou réinitialisation automatique n'est requise par ce guide.

2. Informations à obtenir et à noter

Élément	Valeur / décision
Compte SFTP, hôte et répertoire	login étudiant ; `linserv-info-01.campus.unice.fr` ; confirmer `www/` et l'espace privé hors web
URL LineServ de l'application de production	`https://linserv-info-01.campus.unice.fr/~pj509414/s302/` **à confirmer**
Accès du site	mécanisme réel de restriction, personnes autorisées et disponibilité réseau
Configuration serveur	PHP Web et extensions, réécriture, configuration UserDir, chemins et permissions
Base de production cible	vérifier que le `DB_NAME` réellement configuré pour l'application correspond à la base sélectionnée dans phpMyAdmin/Adminer ; aucune base distincte pour les tests manuels
Connexion applicative	hôte, port ou socket, utilisateur SQL autorisé et méthode de transmission confidentielle
Stockage autorisé	espace privé pour `.env` et journaux `var/`, accessible au processus PHP sans lecture publique
Révision déployée	identifiant Git **et** éventuelles modifications locales, à reporter lors du transfert

Ne jamais écrire un mot de passe ici, dans Git, dans une capture d'écran ou dans l'e-mail de recette. Ne pas supposer que les identifiants phpMyAdmin sont ceux du compte SFTP ou que `localhost`, `127.0.0.1`, le port ou le nom `minishop` conviennent.

3. Préparer la bonne version en local

1. Dans FileZilla, le dossier local de travail de l'utilisateur est `/home/jojokes/Documents/BUT/Projet/s302`. Ne pas supprimer ni remplacer les fichiers personnels ou les modifications locales : notamment `PdoStore.php`, `scripts/seed_mysql.php`, `tests/perf/charge-rb18.sh` et `.githooks/` s'ils existent dans sa copie. Le contenu local peut différer de celui de cette branche ; le constater **avant** tout transfert.
2. Relever l'état sans rien modifier :

```
cd /home/jojokes/Documents/BUT/Projet/s302
git branch --show-current
git rev-parse --short HEAD
git status --short
```

Résultat attendu : nom de branche, identifiant court puis, éventuellement, liste de modifications. Une liste non vide signifie simplement que certains fichiers ne correspondent pas au commit : ne pas les écraser. Pas de `git reset`, `clean`, `checkout --`, `pull` ou fusion automatique dans ce

guide.

1. Vérifier que le guide et les scripts à transférer sont bien présents dans **cette copie**. Si ce n'est pas le cas, récupérer les changements après résolution des éventuels conflits, sans remplacer le travail local. Le commit `HEAD` seul ne décrit pas les modifications non commitées : noter aussi leur présence.

2. Conserver une sauvegarde du projet et, avant la campagne, un export privé/restaurable de l'état de démonstration de la base de production LineServ. Ne pas copier à l'aveugle des données locales ni des secrets vers le serveur ; les mutations de recette utilisent uniquement des données fictives identifiées.

4. Accès SFTP avec FileZilla - préparation, sans publication

1. Ouvrir FileZilla. Dans le gestionnaire de sites, choisir **SFTP - SSH File Transfer Protocol**, hôte `linserv-info-01.campus.unice.fr`, port `22` sauf indication contraire de l'IUT, identifiant étudiant et méthode d'authentification demandée par l'IUT. Vérifier l'empreinte de la clé du serveur auprès de l'IUT lors de la première connexion. Ne pas transmettre le mot de passe à un tiers.

2. À gauche : dossier local indiqué au §3. À droite : constater l'arborescence reçue, repérer `www/` et **demander quel répertoire hors `www/` est autorisé** pour le projet privé. Ne rien supprimer et ne pas remplacer un dossier existant.

3. Si l'IUT autorise cet espace, un dossier de préparation privé distinct peut être créé et les fichiers nécessaires y être copiés après validation de l'équipe. Ne pas téléverser `.git/`, `node_modules/`, `vendor/`, `.env` de la machine locale, sauvegardes, journaux ni données de travail personnelles. Conserver `app/`, `public/`, `.env.example`, les ressources indispensables et la structure du projet. Les documents et SQL restent hors web.

4. **Ne pas encore copier le projet entier dans `www/s302/` ni publier le site.** Une fois le sous-dossier adapté et le point d'entrée configuré, ne publier que le contenu web autorisé, sans `public/mesurer.php` ni `public/gen_volumes.php`, et tester que les fichiers privés ne sont pas accessibles. La disposition finale dépend des possibilités réelles de LineServ ; un simple glisser-déposer n'établit pas cette séparation.

5. Les transferts suivants doivent éviter d'écraser les configurations ou données de production. Garder une sauvegarde et faire relire le plan de mise à jour ; le déploiement du code ne réinitialise pas la base. La restauration de l'état de démo ne se fait qu'après la campagne, selon le §5.5.

5. Base de production LineServ - recette pré-démo et retour à la démo

> **Cible manuelle :** la base réellement utilisée par l'application de production sur LineServ, confirmée dans la configuration privée (`DB_HOST` / `DB_NAME`) et dans phpMyAdmin/Adminer. Ne pas créer une autre base pour la recette. `minishop_test` est réservée aux tests automatisés GitHub CI. Cette campagne pré-démo suivie d'une remise à l'état de démonstration est un plan, pas une

opération attestée.

5.1 Confirmer la cible et les conditions d'entrée

1. Dans la configuration privée réellement lue par PHP, confirmer `APP_ENV=prod`, `STORAGE_DRIVER=sql`, l'hôte `DB_HOST` et le nom `DB_NAME`. Dans phpMyAdmin/Adminer, sélectionner la base et vérifier que l'identité correspond exactement à cette configuration. Ne pas supposer le nom `minishop`, un hôte, un port ou un socket ; si une valeur ne concorde pas ou ne peut pas être vérifiée, **arrêter**.

2. Confirmer avec le propriétaire/IUT que cette base est réservée à MiniShop et qu'elle ne contient que des données fictives de démonstration/test. S'il y a des clients, opérations, données personnelles réelles, autre projet, trafic actif ou objet SQL inexpliqué, ne pas lancer la recette ni réinitialiser : demander une décision explicite au propriétaire.

3. Prévoir une fenêtre pré-démo autorisée et restreindre l'accès HTTP aux seules personnes qui testent. Noter en privé la révision déployée, l'heure de début, l'opérateur et le point de restauration prévu. Les essais sur production peuvent écrire des comptes, commandes, stocks, journaux et sessions.

4. Le document [`13-fiches-recette-uc.md`](13-fiches-recette-uc.md) définit les parcours applicatifs autorisés. Les scénarios nécessitant SQL direct, scripts de purge, rupture forcée du service ou tests de charge ne sont pas à exécuter sur la production ; ils restent automatisés ou sont marqués `BLOQUÉ` dans la campagne manuelle.

5.2 Sauvegarder l'état de démonstration avant les tests

1. Avant toute écriture, exporter la base de production vérifiée dans un emplacement **privé, hors du dépôt et hors du répertoire web**. La sauvegarde doit contenir le schéma, les données, vues, fonctions, procédures et triggers nécessaires au fonctionnement de cette version.

2. Faire confirmer que cette sauvegarde représente bien l'état de démonstration voulu et qu'une procédure autorisée permet de la restaurer **dans la même base de production**. Les paramètres/permissions d'export varient selon les versions de phpMyAdmin/Adminer ; ne pas supposer qu'un export des seules tables contient routines, fonctions ou triggers.

3. Ne pas commencer les scénarios mutatifs si l'export est incomplet, si le stockage de la sauvegarde n'est pas privé, ou si la restauration n'a pas de procédure approuvée. Ne jamais joindre le fichier SQL, les mots de passe, les hashes, les valeurs `.env` ni des données réelles à Git, aux tickets ou aux captures.

Cette sauvegarde est le **point de retour de la campagne**. Elle doit être prise après que l'état de démo a été préparé et vérifié, mais avant d'ajouter les comptes/produits/commandes de test.

5.3 Première initialisation uniquement - base de production attribuée et vide

Cette sous-section ne concerne que le tout premier chargement, lorsque l'IUT a attribué la base de production MiniShop et confirmé par écrit qu'elle est **vide**. Elle ne crée pas de base manuelle distincte et ne sert pas à réinitialiser une base déjà utilisée.

• Le fichier original `sql/01_minishop_schema.sql` contient `DROP DATABASE IF EXISTS minishop`, `CREATE DATABASE`, `USE minishop` et `DROP TABLE IF EXISTS parametre`. Ne jamais l'importer tel quel sur LineServ ni en retirer des lignes à la main. Les fichiers `02` et `03` comportent aussi des `DROP ... IF EXISTS` de routines/triggers ; ne les utiliser que pour cette initialisation explicitement autorisée d'une base vide.

• Depuis la copie locale du dépôt, préparer la version de `01` destinée à une base vide :

```
cd /home/jojokes/Documents/BUT/Projet/s302
python3 scripts/preparer_sql_lineserv.py
```

Le script crée un fichier SQL temporaire local, ne contacte aucun serveur et ne vide ni ne modifie de base. Il retire les instructions de changement/suppression de base de l'original et refuse si le schéma a changé. Ce fichier est destiné **uniquement à une base déjà vide sélectionnée** ; ce n'est pas un outil de reset ni de migration d'une base existante.

• Dans phpMyAdmin, sélectionner la base de production attribuée et confirmée vide, puis importer le fichier `01_schema_pour_base_vide.sql`. Importer ensuite `sql/02_minishop_procedures.sql`, puis `sql/03_minishop_triggers.sql`, dans cet ordre. Les fichiers `02`/`03` contiennent des `DELIMITER` : importer le fichier complet ; si la version phpMyAdmin ou les privilèges refusent l'import, arrêter et demander l'aide de l'IUT plutôt que d'improviser.

• `sql/04_minishop_demo.sql` crée des commandes et modifie les stocks ; ce fichier n'est pas idempotent. Ne l'importer que si ces commandes font explicitement partie de l'état initial de démonstration approuvé, une seule fois après `01`, `02` et `03`. Pour les tests manuels, créer les commandes synthétiques nécessaires via l'application.

• À titre de contrôle en lecture seule, vérifier les compteurs attendus pour cette version : 8 tables, 3 vues, 17 procédures, 1 fonction, 16 triggers ; le jeu `01` fournit 4 catégories, 36 produits et 3 clients. Les commandes dépendent de l'import approuvé de `04`. Toute différence ou import partiel impose un arrêt et un signalement, pas une réimportation à l'aveugle.

5.4 Dérouler la campagne sur LineServ

1. Utiliser uniquement les comptes et données fictifs créés pour le test, clairement identifiés (par exemple références `TEST-UC-...` et adresses `example.test`). Obtenir les identifiants d'accès par canal privé ; ne les inscrire ni dans ce guide ni dans le suivi.

2. Exécuter les scénarios via le navigateur ou des requêtes HTTP contrôlées vers l'application, uniquement sur ces objets synthétiques. Le suivi [`13-suivi-recette-uc.csv`](13-suivi-recette-uc.csv) reste initialisé à `NON JOUÉ` jusqu'à l'exécution réelle ; sa colonne `Base` signifie `APP`/`SPEC`, pas le nom ou l'environnement d'une base SQL.

3. Ne pas exécuter sur LineServ `scripts/run-all-tests.*`, `scripts/run_sql_tests.sh`, `tests/php/sql_integration.php`, `scripts/db_init.php`, `scripts/load_db.sh`, `run.sh --db`, les seeds/fixtures SQL, les outils de purge avec `--executer`, les scripts de volume/charge ou des

requêtes SQL mutatives. Ils ne constituent pas la recette manuelle de production et certains peuvent supprimer ou altérer des données.

4. Si le nom de base, la révision, l'accès autorisé ou la nature synthétique des données devient incertain, arrêter les mutations, noter les scénarios non joués/bloqués et garder l'accès au site restreint.

5.5 Réinitialiser la même base à l'état de démonstration avant la démo

Étape obligatoire après la recette, non exécutée par la rédaction de ce guide. Ne pas ouvrir/reprendre l'accès de démonstration tant que le retour d'état n'a pas été vérifié.

1. Fermer la fenêtre de test et empêcher toute nouvelle écriture applicative.

2. Reconfirmer dans la configuration privée et dans phpMyAdmin/Adminer que la cible est toujours la **même base de production LineServ** que celle sauvegardée avant les tests. Si la cible diffère, ne rien importer.

3. Restaurer dans cette même base l'export complet vérifié au §5.2, selon la procédure de restauration approuvée par l'IUT/le propriétaire. Cette sauvegarde doit être le snapshot de l'état de démonstration pris avant les écritures de test. Ne pas créer une deuxième base, ne pas importer une sauvegarde de travail inconnue et ne pas restaurer si des données non synthétiques doivent être conservées.

4. Comparer en lecture seule les compteurs/objets et l'état fonctionnel au relevé pré-campagne ; vérifier que les données de test (`TEST-UC-...`) ont disparu et que l'application utilise toujours la base restaurée. Avant de rouvrir l'accès de démonstration, confirmer que les comptes de seed aux identifiants publiés ont été neutralisés ou renouvelés par le processus autorisé. Ne publier aucune donnée sensible comme preuve.

5. Si la sauvegarde, la cible ou le résultat de restauration est douteux, garder le site restreint et demander l'aide de l'IUT ; ne pas improviser de `DROP`, d'import ni de réinitialisation.

Avertissement sur les outils du dépôt : `scripts/db_init.php`, `scripts/load_db.sh`, `run.sh --db` et `sql/01_minishop_schema.sql` ne sont pas des commandes de reset de production sûres. Ils ont des hypothèses de nom/privilèges et/ou du DDL destructif. `scripts/preparer_sql_lineserv.py` ne prépare qu'un premier import dans une base vide. Ne les utiliser ni pour nettoyer la base après la recette ni en remplacement de la sauvegarde validée. `minishop_test` est uniquement la base temporaire des tests automatisés CI.

---

6. Configuration applicative - après validation de l'architecture

1. Créer `.env` **côté serveur dans le dossier privé de la racine applicative**, depuis `.env.example`, seulement après avoir déterminé où le processus PHP lit la racine. Ce fichier est ignoré par Git ; ne pas le téléverser sous `www/` ni le joindre à un e-mail. Protéger la lecture selon les permissions et l'identité du processus PHP indiquées par l'IUT : un `chmod 600` arbitraire peut

empêcher le serveur de lire le fichier si les utilisateurs diffèrent.

2. Renseigner `APP_ENV=prod`, `APP_DEBUG=0`, `STORAGE_DRIVER=sql`, `DB_HOST`, `DB_PORT` ou `DB_SOCKET`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` avec les valeurs **réellement attribuées** pour la base de production MiniShop. Utiliser le compte applicatif limité à cette base, avec les seuls droits nécessaires ; les droits de sauvegarde/restauration relèvent du compte autorisé par l'IUT. Les variables d'environnement du processus priment sur `.env` ; vérifier ce point sans publier leurs valeurs.

3. Prévoir les répertoires privés nécessaires aux journaux et s'assurer que PHP peut y écrire, sans les rendre consultables par HTTP. Vérifier les cookies de session en HTTPS, y compris le drapeau `Secure` si un proxy termine TLS. Ne pas déposer de `phpinfo()` public.

4. Une fois l'adaptation du préfixe, du bootstrap et de la réécriture réalisée, faire valider que `app/`, `.env`, `data/`, `var/`, `.git/`, `sql/` ne sont pas servis. Vérifier depuis le navigateur que les URL privées répondent 403/404, **sans afficher ni partager leurs contenus** ; interrompre immédiatement en cas d'exposition et faire retirer l'accès. Une 403/404 seule ne prouve pas toutes les protections.

5. Restreindre l'accès à l'instance de production pendant la fenêtre pré-démo ; utiliser des comptes de test autorisés et fournir leurs identifiants séparément par canal privé. Ne consigner aucun mot de passe dans les documents. Ne pas ouvrir le site aux testeurs tant que la cible, la sauvegarde et les conditions de restauration ne sont pas vérifiées.

7. Contrôles sur l'instance LineServ avant invitation

Sur l'instance de production LineServ, pendant la fenêtre pré-démo autorisée, utiliser uniquement des comptes et données fictifs. Avant la recette, confirmer `DB_HOST`/`DB_NAME` en privé, l'identité de la base sélectionnée, l'accès restreint et la sauvegarde restaurable. Relever l'URL, la date et la révision déployée, sans noter de secret ni de nom de base dans une preuve publique.

- [ ] L'accueil, les images locales (visuel informatif en fiche produit, vignettes décoratives en liste) et CSS/JS se chargent ; cliquer catalogue, recherche et fiche produit ne sort pas de `/~pj509414/s302/`.
- [ ] Formulaires et navigation conservent le préfixe ; les URL directes et rechargements fonctionnent ; une route inconnue donne une 404 sans détail interne.
- [ ] Inscription, connexion, panier (avec et sans JavaScript), commande simulée, historique et annulation client en 2 étapes (avec conservation des lignes en historique) fonctionnent avec les comptes de recette.
- [ ] Après connexion admin autorisée : listes, mode par défaut « Ajouter / retirer » (`DELTA`) sur `/admin/stocks`, modification d'un produit fictif, filtres de dates en fuseau local et évolution/annulation en 2 étapes d'une commande de test (avec retour de stock avant expédition et sans retour automatique si `EXPEDIEE`) fonctionnent ; aucune action ne modifie d'autres projets.
- [ ] Sur `/preferences` (sans connexion requise) : vérifier la bascule intégrale `Français` <-> `English` sur les pages publiques, client et admin, le choix manuel ou par géolocalisation locale sans

requête réseau externe du fuseau horaire, et la réinitialisation.

• [ ] Sur `/mentions-legales` : contrôler l'affichage exact des co-éditeurs étudiants et des coordonnées de l'hébergeur IUT.

• [ ] Répéter un parcours avec deux sessions isolées ; vérifier qu'aucune commande d'un autre compte n'est exposée (404).

• [ ] Parcourir sur ordinateur au clavier (`Tab`, `Maj+Tab`, `Entrée`) et vérifier une page publique et une page connectée avec WAVE (§8) - les tests responsive multi-tailles sont exclus de la campagne manuelle et couverts par la CI automatisée.

• [ ] Pas d'accès public aux fichiers privés ou aux outils de développement (`public/mesurer.php`, `public/gen_volumes.php`) ; débogage désactivé ; comptes de démonstration non exposés.

• [ ] Consigner les résultats réels et anomalies dans [la fiche](13-fiches-recette-uc.md) et [le suivi CSV](13-suivi-recette-uc.csv), pas dans ce guide comme s'ils étaient déjà validés. Après la recette, restaurer la même base à l'état de démonstration selon le §5.5 avant de rouvrir l'accès.

Problèmes fréquents : **404 sur `/catalogue` ou CSS absent** -> préfixe, réécriture et liens non adaptés ; **500 dès l'accueil** -> chemin du bootstrap, droits de lecture, version PHP ou base ; **connexion DB refusée** -> coordonnées et droits attribués, sans montrer le mot de passe ; **import SQL incomplet** -> routines, délimiteurs ou privilèges, à constater sans réinitialiser. Ne pas activer le debug public pour obtenir un message ; voir les journaux privés avec l'accord de l'IUT.

8. Logiciels pour l'accessibilité (et mémo optionnel hors campagne)

WAVE : repérer les problèmes d'accessibilité

[WAVE de WebAIM](https://wave.webaim.org/extension/) est une extension gratuite pour Firefox, Chrome et Edge. Elle analyse la page **rendue**, donc fonctionne avec un site PHP ; l'extension peut aussi analyser une page d'intranet ou une page où l'on est connecté. Selon WebAIM, l'analyse de l'extension s'effectue dans le navigateur, sans envoyer le contenu au serveur WAVE. Utiliser l'extension plutôt que le formulaire en ligne pour les pages privées.

1. Installer l'extension depuis le lien officiel et ouvrir l'URL LineServ réellement validée.

2. Sur l'accueil, cliquer l'icône WAVE. Relever les **Errors**, erreurs de contraste et **Alerts** : titre, description, élément et capture expurgée. Une alerte demande une vérification humaine, elle n'est pas forcément un défaut.

3. Recommencer pour catalogue, produit, panier, inscription, commande et les pages connectées/admin autorisées, y compris quand un message d'erreur est affiché. Une analyse ne couvre que la page et l'état affichés.

4. Tester en plus au clavier (`Tab`, `Maj+Tab`, `Entrée`, `Espace`, `Échap`) : focus visible, ordre logique, liens et formulaires utilisables.

Limite : WAVE détecte une partie des problèmes seulement. Ni absence d'erreurs ni un score ne prouve une conformité WCAG ou RGAA. Un audit humain approfondi et, le cas échéant, une

évaluation experte restent nécessaires. Source : [aide officielle WAVE](https://wave.webaim.org/help).

Note hors campagne manuelle : pourquoi le responsive n'est pas rejoué à la main

Par décision d'équipe, la vérification responsive multi-largeurs (`320`, `375`, `768`, `1440 px`) a été **retirée des suites de tests manuels** car trop longue à dérouler sur toutes les pages ; elle est exécutée automatiquement à chaque push par `tests/php/responsive_contract.php` et `tests/node/responsive_navigateur.mjs` (Chromium headless en CI). En cas d'anomalie visuelle ponctuelle signalée sur un écran particulier, le mode appareil de Chrome/Chromium (`F12` puis `Ctrl+Maj+M`, [documentation Chrome Device Mode](https://developer.chrome.com/docs/devtools/device-mode)) reste utilisable uniquement à titre de diagnostic ciblé, sans campagne complète exigée.

9. Documents et communication

• [E-mail prêt à personnaliser pour la collègue](15-email-recette-lineserv.md), à envoyer seulement quand URL, accès, sauvegarde et procédure de retour à la démo sont réellement prêts.

• [`docs/22-procedure-validation-staging.md`](22-procedure-validation-staging.md) reste une procédure distincte de staging isolé ; elle ne remplace pas cette campagne manuelle sur la base de production LineServ.

• [Fiches des 14 UC](13-fiches-recette-uc.md) : actions, attendus, risques et protocole.

• [Tableau de suivi CSV](13-suivi-recette-uc.csv) : `OK`, `KO`, `BLOQUÉ`, `NON JOUÉ` ou `NON APPLICABLE`, preuves et résultats réels.

• [Cahier des charges](01-cahier-des-charges-MiniShop.md), [tests et validation](02-document-tests-validation.md), [information juridique de la maquette](12-information-juridique.md), [étude de design](DA.md).

• La maquette ne réalise ni paiement ni livraison ; l'annulation d'une commande n'est pas une fonction légale de rétractation. Aucune vente ni donnée personnelle réelle.

Sources : [phpMyAdmin - import dans une base sélectionnée](https://docs.phpmyadmin.net/en/latest/import_export.html), [WAVE](https://wave.webaim.org/extension/), [Chrome Device Mode](https://developer.chrome.com/docs/devtools/device-mode). Les indications propres à LineServ viennent de l'exemple de l'utilisateur et restent à confirmer sur place.